GeoCampus System Improvements

Architectural Roadmap & Security Hardening Specifications (Prototype to Production)

1. Executive Reality Check & Objectives

While the initial GeoCampus MVP prototype successfully demonstrates hybrid attendance, server Haversine calculations, and dynamic QR tokens, browser-based storage models introduce inherent security limitations. To elevate the platform from a 6/10 prototype to a robust **9.5/10 enterprise-grade production application**, this document outlines the systematic removal of client-side trust in favor of a secure, server-authoritative architecture.

Current Prototype Limitation

Credentials and session roles stored in browser `localStorage` or plaintext arrays allow potential client-side tampering and prompt token hijacking.

Production Target State

A decoupled architecture featuring Node.js/Express backends, SQLite/PostgreSQL databases, bcrypt hashing, and cryptographically signed JWTs.

2. Critical Security & Authentication Fixes

Eliminating plaintext credentials and shifting trust boundaries entirely to the server-side environment.

Vulnerability Area	Prototype Risk	Production Solution
Plaintext Passwords	Visible in frontend JavaScript array sources.	Remove user arrays from client; enforce `bcrypt` hashing (cost factor 12) in database.
Role Trust	Users can overwrite role state in `localStorage`.	Server issues signed JWTs; middleware verifies roles on every protected API call.
Client Check-In	Check-in validation and logging executed entirely in browser script.	Server validates tokens, expiration, duplicates, and Haversine distance.

3. Geofencing & Anti-Spoofing Hardening

GPS spoofing and proxy check-ins are mitigated through strict server-side geometry rules and telemetry evaluation heuristics.

Component	Improvement Strategy	Security Impact
Strict Radius Enforcement	Drop radius-plus-accuracy addition. Enforce strict distance check (`distance ≤ campus.radius`).	Prevents boundary expansion under weak satellite signals.
GPS Accuracy Threshold	Automatically reject check-ins reporting accuracy values greater than 30m–50m.	Filters out low-confidence or artificially manipulated location fixes.
Impossible Travel Check	Evaluate timestamp gaps between consecutive student check-ins across distant sectors.	Flags accounts sharing credentials or spoofing coordinates unrealistically.
Device Fingerprinting	Track device session identifiers to detect multi-student logins originating from a single hardware hash.	Mitigates proxy attendance and classroom device-sharing rings.

4. File Management & Workflow Enhancements

Handling student On-Duty (OD) proof uploads securely and ensuring multi-device session sync for all campus participants.

Server-Side OD File Handling

Integrate `multer` middleware with strict file type filters (PNG, JPEG, PDF) and 2MB size caps, storing files securely outside public web roots.

Token Auto-Rotation & Sync

Faculty-generated session tokens rotate automatically every 45 seconds on the server. The backend retains a grace window for previous tokens to absorb network latency.

5. Production Relational Database Schema

Migrating from flat JSON files to structured relational tables (SQLite/PostgreSQL) with strict indexes and constraints.

> **Core Entities & Constraints**
> **Users:** ID, Name, Role (Student/Faculty/Admin), Password Hash.
> **Sessions:** Auto ID, Token (Unique), Course, Faculty ID, Expires At.
> **Attendance:** Session ID + User ID (Unique Composite Index for verified status), Lat, Lon, Distance, Accuracy, Status.
> **Config & OD Requests:** Campus Geofence Center (Lat/Lon/Radius), OD Proof Path, Status (Pending/Approved).

6. Recommended Implementation Sequence

- **Phase 1:** Establish Node.js backend structure, SQLite/Prisma database, and migration scripts.
- **Phase 2:** Implement secure `/api/login` with `bcrypt` and JWT middleware protection.
- **Phase 3:** Migrate Haversine calculations and session creation to the server authority.
- **Phase 4:** Apply XSS escape utilities (`esc()`) and Helmet security headers.
- **Phase 5:** Integrate QR scanner libraries, token auto-rotation, and production HTTPS deployment.

7. Conclusion

Implementing this roadmap successfully addresses all prototype security gaps, protects institutional data integrity, and achieves an estimated security and architecture score of **9.5/10**.